

Shohoj Lipi

Enhancing Bangla Text Accessibility for Dyslexic Readers
through LLM-Driven Simplification and Adaptive Typography

সহজ লিপি

Full 7-Day Execution Plan | SciBlitz AI Challenge 2026 | IEEE CUET

Track C — Education & Accessibility

Member A	Member B	Member C	Member D
AI/NLP Lead	Frontend Lead	Backend/Eval Lead	Data & Report Lead

1. Project Overview

1.1 Problem Statement

An estimated 10–15% of Bangladeshi school-age children experience reading difficulties including dyslexia. Standard Bangla typography is particularly challenging for these readers: dense conjunct consonants (যুক্তবর্ণ), tight inter-character spacing, serif-heavy fonts, and long unbroken sentences create extreme visual crowding. No accessible, AI-powered Bangla reading tool exists today.

Students, parents, and teachers have no way to transform a textbook paragraph, a newspaper article, or a story into a form that is easier to decode — resulting in children being labeled 'slow learners' rather than receiving appropriate support.

1.2 Solution: Shohoj Lipi

Shohoj Lipi is a web application where a user pastes any Bangla text. The AI pipeline then:

- Predicts the readability level of the input (grade 1–12) using a fine-tuned BanglaBERT classifier
- Simplifies complex sentences via an LLM (Qwen2.5-Instruct or GPT-4o-mini) — shorter clauses, simpler vocabulary drawn from a 3,396-word easy-word list
- Renders the output in a dyslexia-optimised visual layout: larger font, generous line height, alternating line shading, inter-word spacing boost, and syllable-boundary highlighting
- Optionally reads the text aloud (Bangla TTS) so users can follow along
- Shows a before/after readability score so teachers can quantify the transformation

1.3 Why This Is Novel

No deployed Bangla dyslexia-reading tool exists. The closest published work is the AAAI 2021 BengaliReadability paper (Chakraborty et al.), which provides only a readability classifier — no simplification, no visual rendering, no user-facing product. Shohoj Lipi stacks the classifier on top of LLM-based simplification and a carefully designed accessible rendering layer, combining three independent research threads for the first time in Bangla.

1.4 Track Alignment

Criterion	Shohoj Lipi's Answer
Innovation (25%)	First Bangla dyslexia-accessibility product; novel combination of readability classification + LLM simplification + accessible rendering
Technical Implementation (25%)	Fine-tuned BanglaBERT + prompted LLM + quantitative before/after readability scoring
Real-World Impact (20%)	Directly serves children with reading difficulties in Bangladesh — a population with no existing tool support
Demo Quality (20%)	Live web app — paste text, instant visual transform; before/after readability score shown side by side
Presentation (10%)	Strong narrative: a child with dyslexia deserves to read their own textbook

2. Dataset & Evaluation Resources

2.1 Primary Dataset — BengaliReadability (AAAI 2021)

Source: github.com/tafseer-nayeem/BengaliReadability — public, MIT-licensed, download on Day 1.

Component	Size	Purpose in PaathSahay
Document-level corpus	618 documents	Fine-tune BanglaBERT readability classifier (12 grade levels → 3 tiers: Easy / Medium / Hard)
Sentence-level corpus	96,000+ sentences	Train sentence-complexity tagger that drives per-sentence simplification decisions
Easy-word list	3,396 words	Vocabulary constraint for LLM simplification prompts — 'prefer words from this list'
Consonant-conjunct algorithm	341 validated words	Compute CC-density (conjunct count per 100 chars) — a key readability feature and evaluation metric
Pronunciation dictionary	67,000+ words	Syllable segmentation for TTS and syllable-boundary highlights in the rendered output

2.2 Supplementary Text Sources (for simplification input corpus)

- NCTB-QA dataset (arXiv 2025) — 87,805 QA pairs from 50 NCTB textbooks: ready source of grade-level Bangla passages to run through the simplification pipeline
- UDDIPOK dataset (Jahangirnagar University, 2023) — 270 passages from textbooks, newspapers, exam questions: diverse domain coverage
- BANS (Bengali Abstractive News Summarization) corpus — newspaper articles usable as 'hard' input texts for the simplification demo

2.3 Evaluation Metrics — What Goes in the Report's Results Section

Metric	What It Measures	How to Compute
Classifier Accuracy / F1	Readability grade prediction quality	sklearn metrics on held-out 20% of the 618-document corpus; compare to AAAI 2021 paper baseline
Sentence-level Accuracy	Simple vs. Complex sentence classification	BanglaBERT fine-tuned on 96K sentence corpus; baseline from the same paper available
Readability Score Delta	Did simplification actually lower reading difficulty?	Run classifier on original vs. simplified text; report mean grade-level drop across 100 test passages

CC-Density Reduction	Conjunct-consonant crowding reduced?	Apply the paper's CC algorithm to original vs. simplified; report mean CC count per 100 chars
BERTScore (Semantic Fidelity)	Does simplified text preserve meaning?	Compare simplified vs. original with BERTScore using multilingual BERT; target F1 ≥ 0.82
Human Readability Rating	Does it feel easier to read?	5-point Likert scale rated by 3–5 native Bangla readers on 30 sample transformations (qualitative)

3. System Architecture

3.1 Pipeline Overview

PaathSahay is a 4-stage sequential pipeline. Each stage is independently testable, which is important for parallel development across 4 team members.

#	Stage	What Happens	Owner
1	Input & Preprocessing	Accept raw Bangla text → normalise Unicode (NFC) → sentence-tokenise using bnlp → compute CC-density, word count, avg sentence length as input features	Member C (Backend)
2	Readability Classification	Fine-tuned BanglaBERT assigns each sentence a complexity label (Simple/Complex) and gives the full passage a grade-tier (Easy / Medium / Hard) with a numeric readability score	Member A (AI/NLP)
3	LLM Simplification	Complex sentences are sent to LLM (Qwen2.5-3B-Instruct locally OR GPT-4o-mini via API) with a structured prompt: shorten, avoid rare conjuncts, prefer the 3,396 easy words; simple sentences pass through unchanged	Member A (AI/NLP)
4	Accessible Rendering	Simplified text is rendered in a dyslexia-optimised layout: SiyamRupali / Nirmala UI font, 20px+ size, 1.8 line-height, syllable-boundary dots, alternating-line tint, adjustable settings; TTS via gTTS/Coqui; before/after readability scores displayed side by side	Member B (Frontend)

3.2 Technology Stack

Layer	Tool / Library	Reason for Choice
NLP Preprocessing	bnlp (pip install bnlp-toolkit)	Best Bangla tokeniser/sentence splitter; also provides word2vec and POS tagger if needed
Readability Classifier	BanglaBERT (csebuetnlp/banglaBERT) via HuggingFace Transformers	Best pretrained Bangla language model; fine-tune on BengaliReadability corpus in ~2 hours on Colab T4

LLM Simplification	Qwen2.5-3B-Instruct (local, 4-bit GGUF) or GPT-4o-mini API	Qwen2.5 has strong Bangla support; 3B fits on Colab free tier; API fallback if quality insufficient
TTS	gTTS (Google TTS, free) + pydub for playback	Simplest Bangla TTS with zero setup; can upgrade to Coqui-TTS for offline
Frontend UI	React 18 + Tailwind CSS + Vite	Tailwind enables rapid accessible-UI styling (custom dyslexia CSS tokens); Vite gives instant HMR; React component model maps cleanly onto the 3-panel layout
Backend API	FastAPI + Uvicorn	Lightweight async Python API; CORS configured so the React SPA can call it from the browser; separates AI logic from UI cleanly
Evaluation	scikit-learn + bert-score + numpy	Standard accuracy/F1 + semantic fidelity (BERTScore) to fill Results section
Deployment — Frontend	Vercel (React SPA, free tier)	One-click GitHub → Vercel deploy; automatic HTTPS; never sleeps unlike HF Spaces; public URL permanent
Deployment — Backend	Render free tier (FastAPI container)	Hosts the AI pipeline separately from the frontend; free tier sufficient; Dockerfile included in repo
Version Control	GitHub (public repo, commits from Day 1)	Required by rulebook; commit history must show activity during build window

4. Feature List

4.1 Core Features (must ship by Day 6)

Feature		Details	Owner
F1	Text Input & Normalisation	Paste box, Unicode NFC normalisation, character limit warning at 2,000 chars	Member C
F2	Readability Score (Before)	Classifier runs on input; displays grade tier + numeric score badge	Member A
F3	AI Simplification	Sentence-by-sentence: complex → simplified; simple → pass through; runs in <5 sec for 300-word input	Member A
F4	Readability Score (After)	Same classifier on simplified output; side-by-side before/after badge shows score delta	Member A
F5	Accessible Rendered Output	SiyamRupali font via @font-face, Tailwind custom class 'dyslexia-body' (text-xl, leading-loose, tracking-wide), alternating row tint via Tailwind even:bg-blue-50	Member B
F6	Font & Layout Controls	React state-driven controls: range input for font size (16–30px via Tailwind arbitrary values), toggle switches for line-shade and syllable dots, all updating CSS custom properties in real time	Member B
F7	Text-to-Speech	Play/pause button reads simplified text aloud via gTTS (Bangla); audio generated server-side	Member C
F8	Copy & Download	Copy simplified text to clipboard; download as .txt	Member B

4.2 Stretch Features (if time permits after Day 6)

- Syllable-boundary highlighting (insert a faint dot between syllables using the pronunciation dictionary)
- Sentence-level confidence colour coding (green = simple already, orange = simplified, red = could not simplify sufficiently)
- Side-by-side diff view (original vs. simplified, changed words highlighted)
- User profile save (remember font size preference across sessions)

5. 7-Day Execution Plan

Each day entry shows: Objectives → Tasks per member → End-of-day deliverable (what is committed to GitHub).

DAY 1 — Foundation & Data (July 2)

Objectives

- Repo created, all tools installed, all datasets downloaded and verified
- Every member has a running environment and has committed at least one file

Member	Tasks
A — AI/NLP	Clone BengaliReadability repo → audit document corpus (618 docs) and sentence corpus (96K) → confirm splits → set up Colab notebook with BanglaBERT loaded → verify fine-tuning loop runs on 10 sample rows
B — Frontend	Scaffold React + Vite project (npm create vite@latest shohoj-lipi -- --template react) → install Tailwind CSS (v3) and configure tailwind.config.js → create 3-panel layout component (InputPanel ControlPanel OutputPanel) as static skeleton → add SiyamRupali.ttf to /public/fonts and configure @font-face in index.css → push to GitHub
C — Backend	Set up FastAPI project → /health endpoint → Bangla sentence tokeniser test using bnlp on 5 sample texts → write Unicode NFC normaliser → confirm gTTS produces audio for a Bangla sentence
D — Data/Report	Download NCTB-QA and UDDIPOK datasets → extract 200 diverse passages (50 easy / 50 medium / 50 hard / 50 news) as evaluation corpus → begin GitHub README with project description, dataset sources, and team roles

End-of-Day Commit: GitHub repo (shohoj-lipi-dyslexia) with /data folder (BengaliReadability + 200 eval passages), /backend skeleton, /frontend React+Vite skeleton with Tailwind configured, README.md

DAY 2 — Classifier Training (July 3)

Objectives

- BanglaBERT readability classifier trained and evaluated on held-out set
- Sentence-level complexity tagger functional

Member	Tasks
A — AI/NLP	Fine-tune BanglaBERT on 618-document corpus for 3-class grade-tier prediction (Easy/Medium/Hard) — 80/20 train/val split, 5 epochs, AdamW lr=2e-5 → log accuracy, F1 per class → save best checkpoint → repeat for sentence-level binary tagger on 96K corpus (can sub-sample 30K for speed)
B — Frontend	Build InputPanel component (Tailwind: textarea with rounded-xl border-2, char counter as text-sm text-gray-400, submit button with bg-navy hover state) → build OutputPanel with placeholder text in accessible layout → define Tailwind custom tokens in tailwind.config.js: fontFamily.bangla, lineHeight.reading (1.8), letterSpacing.bangla (0.02em), wordSpacing utility via plugin → wire CSS custom property --font-size to a React state variable

C — Backend	Write /classify endpoint that accepts raw Bangla text → calls tokeniser → returns readability tier + numeric score (computed from CC-density + avg sentence length features) → unit test with 10 passages → wire gTTS into /tts endpoint returning MP3 bytes
D — Data/Report	Run AAAI 2021 CC-density algorithm on the 200 eval passages → record baseline CC counts per passage → start Results table in report doc → write Problem Statement and Related Work sections of report

End-of-Day Commit: Trained BanglaBERT checkpoint in /models, /backend /classify endpoint passing tests, accessibility CSS in /frontend, baseline CC table in /docs

DAY 3 — LLM Simplification Pipeline (July 4)

Objectives

- LLM-based simplification working end-to-end on test sentences
- Frontend connected to backend classify endpoint

Member	Tasks
A — AI/NLP	Design simplification prompt (system: role as Bangla language accessibility expert + easy-word list snippet; user: 'Simplify this sentence: {sentence}') → test on 20 complex sentences from eval corpus → compare Qwen2.5-3B (local Colab) vs GPT-4o-mini (API) on quality → write /simplify endpoint wrapping chosen LLM → add sentence-level routing: if sentence complexity tag = Simple, skip LLM
B — Frontend	Connect textarea submit → POST to /classify → display readability badge (colour-coded: green Easy, orange Medium, red Hard) → add loading spinner → display result in accessible layout panel; wire font-size slider and line-shade toggle to CSS variables
C — Backend	Integrate BanglaBERT checkpoint into /classify (replace feature-based placeholder) → add /simplify endpoint that calls simplification pipeline → add CORS headers so frontend can call from browser → add input validation (reject non-Bangla / empty)
D — Data/Report	Run simplification on 50 'Hard' eval passages → record readability score before and after for each → fill results table → write Methodology section of report (pipeline diagram + prompt template)

End-of-Day Commit: /simplify endpoint passing 20 test cases, frontend wired to /classify showing badge, initial Results table with 50 rows in /docs

DAY 4 — Full Pipeline Integration (July 5)

Objectives

- Complete end-to-end flow working in browser: paste → classify → simplify → render → TTS
- Before/after readability score visible to user

Member	Tasks
A — AI/NLP	Run BERTScore evaluation on 50 simplified passages (original vs. simplified, multilingual-bert) → record F1 scores → iterate on prompt if BERTScore <

	0.80 → fine-tune prompt with 5 few-shot examples for worst-performing passages
B — Frontend	Wire the full flow: submit → /classify (before badge) → /simplify → re-run /classify on simplified output (after badge) → render simplified text in accessible layout → show score delta ('Difficulty reduced from Grade 9 → Grade 4') → add TTS play button calling /tts
C — Backend	Chain classify + simplify in a single /process endpoint returning {before_score, before_tier, simplified_text, after_score, after_tier} — reduces frontend API calls from 3 to 1 → add response caching (lru_cache) for repeated identical inputs → stress test with 10 concurrent requests
D — Data/Report	Run human readability rating: recruit 3 native Bangla speakers (teammates + 1 classmate) → rate 30 before/after pairs on 5-point Likert scale → compute mean and inter-rater agreement → add to Results section → write Evaluation section of report

End-of-Day Commit: Working /process endpoint, full pipeline wired in browser, BERTScore results and human rating data in /docs

DAY 5 — Evaluation, Polish & Syllable Features (July 6)

Objectives

- All quantitative evaluation metrics computed and written up
- Stretch features (syllable dots, diff view) implemented if time allows

Member	Tasks
A — AI/NLP	Compute final classifier accuracy/F1 on held-out doc corpus → compute mean grade-level drop across all 200 eval passages → compile full evaluation table → deliver final numbers to Member D for report
B — Frontend	Implement syllable-boundary dot insertion using the pronunciation dictionary (parse dict → split word on syllable boundary → insert U+00B7 midpoint dot) → add diff highlight (changed words in light amber) → responsive layout for mobile screens (important for accessibility)
C — Backend	Add /syllabify endpoint that looks up words in pronunciation dictionary and returns syllable-split version → add error handling for all endpoints with clean JSON error responses → write docstrings and inline comments for every function (Model Card / Data Card prep)
D — Data/Report	Compile complete Results section with all 6 metrics + table + figures (bar chart: before/after score per passage group; confusion matrix for classifier) → write Limitations & Future Work section → assemble full 8-page report draft

End-of-Day Commit: Syllable endpoint, complete eval numbers in /docs, 8-page report draft

DAY 6 — Deployment & Demo Hardening (July 7)

Objectives

- App live on Vercel with a public URL — the single most critical milestone
- Demo video recorded

Member	Tasks
A — AI/NLP	Export BanglaBERT model to HuggingFace Hub (private → public after verification) → test inference via Hub API → add 3 preloaded example inputs (easy, medium, hard text) as 'Try an example' buttons to reduce cold-start demo friction
B — Frontend	Run npm run build → deploy React SPA to Vercel via GitHub integration (vercel.com → import repo → one-click deploy) → test all features on the live Vercel URL → fix any spacing/font issues that render differently in hosted environment → update page title, meta description, and README with team name
C — Backend	Containerise FastAPI backend (Dockerfile) → deploy to Render free tier → configure CORS to allow the Vercel frontend domain → run 20 manual test cases on the live URL → fix any cold-start issues (add a /warmup endpoint called on startup)
D — Data/Report	Record 3–5 minute demo video: (1) paste a NCTB textbook paragraph, (2) show readability score drop, (3) show accessible rendering, (4) play TTS, (5) adjust font size → upload to YouTube unlisted → write Model & Data Card (1-page PDF)

End-of-Day Commit: Live public URL confirmed, demo video link in README, Model & Data Card PDF in /docs

DAY 7 — Report Finalisation & Submission (July 8)

Objectives

- All 5 submission deliverables packaged and submitted before 11:59 PM BST

Member	Tasks
A — AI/NLP	Review and proofread Methodology + Results sections of report → verify all numbers are final and match eval notebook output → push final model card
B — Frontend	Final QA on live URL: test all 8 features, test on mobile, test TTS → fix any regressions → ensure app does NOT sleep before July 12 (visit it manually morning of July 9 as per rulebook Section 9.3)
C — Backend	Make GitHub repo public → verify commit history shows activity from Day 1 → write final README (tech stack, setup instructions, dataset sources, team) → add badges (Vercel deploy, Python version)
D — Data/Report	Final proofread of full 8-page report → export to PDF → confirm all 5 deliverables: (1) Live URL, (2) Report PDF, (3) GitHub link, (4) YouTube demo link, (5) Model & Data Card PDF → submit via official form

Submission Checklist: Live URL ✓ | Report PDF ✓ | GitHub (public, with commits) ✓ | Demo video (YouTube unlisted) ✓ | Model & Data Card PDF ✓

6. Member Role Summary

Member	Role	Full Ownership
Member A	AI / NLP Lead	BanglaBERT fine-tuning (readability classifier + sentence tagger), LLM simplification pipeline and prompt engineering, BERTScore evaluation, model export to HuggingFace Hub
Member B	Frontend Lead	React + Tailwind CSS UI (all panels), accessible CSS layout, font/line controls, TTS play button, syllable dot rendering, diff highlight, Vercel deployment, demo QA
Member C	Backend Lead	FastAPI server (/classify, /simplify, /process, /tts, /syllabify endpoints), bnlp preprocessing, Unicode normaliser, gTTS integration, caching, Dockerfile, Render deployment
Member D	Data & Report Lead	Dataset download and curation (BengaliReadability + 200 eval passages), CC-density baseline computation, human evaluation study, all 6 evaluation metrics, full 8-page report, demo video recording, Model & Data Card

All members are expected to contribute GitHub commits throughout (Day 1–7). Member D takes the submission lead on Day 7 but all four verify deliverables before the 11:59 PM deadline.

7. Risk Register & Mitigations

Risk	Severity	When	Mitigation
LLM output quality poor for complex Bangla conjuncts	High	Day 3	Switch to GPT-4o-mini API (better Bangla handling); add few-shot examples to prompt; fall back to rule-based sentence shortening if needed
BanglaBERT fine-tune fails on small corpus (618 docs)	Med	Day 2	Use data augmentation (back-translation via BanglaT5 bn2en → en2bn) to 3× corpus size; or use feature-based classifier (XGBoost on CC-density + sentence-length features) as backup
HuggingFace Space sleeps / crashes on demo day	High	Day 6+	Keep a pre-recorded demo video ready (rulebook Section 9.3 explicitly advises this); Vercel never sleeps so cold-start latency is not a concern; trigger a fresh Vercel redeploy from the dashboard if any issue arises before demo day
Bangla font does not render correctly in browser on Vercel-hosted app	Med	Day 6	Bundle SiyamRupali.ttf as a local file in the repo and load via CSS @font-face rather than relying on system font; test on Day 5 pre-deployment
gTTS rate-limited or offline on demo	Low	Day 7	Pre-generate TTS audio for the 3 example inputs and cache them; TTS is a stretch feature — demo video can show it without relying on live call
Member unavailable for a day	Med	Any	Each critical path task (train model, deploy, submit) has a named backup; daily 15-min standup via WhatsApp/Discord to surface blockers early

8. Project Report Outline (8 Pages)

Required structure per rulebook Section 7.1: Problem → Solution → Methodology → AI/ML Approach → Results → Limitations & Future Work.

Se c	Title	Pages	Key Content
1	Problem Statement	0.5	Bangla dyslexia prevalence, lack of existing tools, sample passage showing density of conjuncts
2	Related Work	0.5	AAAI 2021 BengaliReadability paper; LLM text simplification literature; dyslexia font research (Good Fonts for Dyslexia, 2013)
3	Proposed Solution	0.5	PaathSahay overview, target users (students / parents / teachers), 4-stage pipeline diagram
4	Methodology & AI/ML Approach	2.5	BanglaBERT fine-tuning details (architecture, hyperparameters, training split); LLM prompt template; sentence routing logic; dataset description (BengaliReadability corpus stats); CC-density computation
5	Results	2.5	Table: classifier accuracy/F1 vs. AAAI 2021 baseline; bar chart: mean grade-level drop across 200 passages; BERTScore distribution; CC-density before/after; human Likert ratings (mean + std)
6	Limitations & Future Work	1	LLM hallucination risk for rare vocabulary; no formal clinical validation with dyslexic users; TTS naturalness; future: teacher dashboard, PDF input, clinical user study
7	References	0.5	BengaliReadability (AAAI 2021), BanglaBERT (NAACL 2022), NCTB-QA (arXiv 2025), Good Fonts for Dyslexia (CHI 2012), Qwen2.5 tech report

9. Key Links & Resources

Resource	URL / Source
BengaliReadability Dataset + Code	github.com/tafseer-nayeem/BengaliReadability
BanglaBERT (HuggingFace)	huggingface.co/csebuetnlp/banglaBERT
NCTB-QA Passages	arXiv:2603.05462 — contact authors for dataset access

bnlp Toolkit (Bangla NLP)	pip install bnlp-toolkit github.com/sagorbrur/bnlp
Qwen2.5-3B-Instruct (GGUF)	huggingface.co/Qwen/Qwen2.5-3B-Instruct-GGUF
SiyamRupali Font	ekushey-project.org / include in repo as /assets/SiyamRupali.ttf
Vercel (React SPA)	vercel.com → import GitHub repo → one-click deploy → free tier (no sleep, automatic HTTPS)
bert-score (pip)	pip install bert-score use model='bert-base-multilingual-cased'

PaathSahay — পাঠ সহায় / SciBlitz AI Challenge 2026

Every child deserves to read their own textbook.